Projektuppgift

Programmering i C#.NET

Det försvunna kungariket

Lina Wiklund



Mittuniversitetet

MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

Det försvunna kungariket
Lina Wiklund 2026-10-28

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Lina Wiklund, liwi2506@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Sammanfattning

I den här rapporten så får läsaren information om hur spelet *Det Försvunna Kungariket* planerades och utvecklades. Läsaren får se vad som fungerade bra med utvecklingen och vad utvecklaren hade gjort annorlunda. Hela spelet är gjort med programmeringsspråket C# och läsaren får lära sig hur C# kan användas för att göra enkla konsolapplikationer.

Förord

Något av mina intressen är att spela spel och det intresset kommer från min äldre bror. Det är något som jag gör för att slappna av på kvällarna när barnen har gått och lagt sig och jag vill ha lite tid för mig själv. Därför valde jag att skapa ett lättare spel som är av typen textbaserat RPG-spel.

Innehållsförteckning

Sammanfattning **iii**

Förord **iv**

1 Inledning **5**
1.1 Bakgrund 5
1.2 Problemdefinition 5
1.3 Syfte 5
1.4 Mål och avgränsningar 6
1.5 Författarens bidrag 6

2 Bakgrundsmaterial **7**
2.1 C# 7
2.2 .NET 7
2.3 Variabler och datatyper 7
2.4 Villkorssatser 8
2.5 Loopar 9
2.6 Användarinmatning och utskrifter 10
2.7 Logiska uttryck 10
2.8 Sammanfattning av teorin 11

3 Metod **12**

4 Konstruktion **18**

5 Resultat **22**

6 Slutsatser **23**

Källförteckning **24**

1 Inledning

1.1 Bakgrund

I kursen Programmering i C#.NET ingår ett projekt där kunskaper från kursen ska användas för att utveckla en applikation i C# med hjälp av ramverket .NET. Projektet ger möjlighet att tillämpa programmering i ett större sammanhang och att arbeta med planering, konstruktion, testning och dokumentation av en applikation.

Jag valde att utveckla ett textbaserat rollspel som heter *Det Försvunna Kungariket.* Spelet är ett konsolprogram där spelaren får utforska ett kungarike, möta fiender, hitta skattkistor och samla guldmynt. Guldmynten kan sedan användas för att köpa föremål i en butik. Spelets slutmål är att besegra en drake som fungerar som en slutboss.

Jag valde ett spel som projekt eftersom det ger möjlighet att använda flera grundläggande programmeringstekniker i en och samma applikation. Spelet innehåller bland annat variabler, villkorssatser, loopar, användarinmatning och beräkningar. Dessa delar behövs för att hantera spelarens val, hälsa, guld, strider och spelets olika händelser.

1.2 Problemdefinition

Utmaningen i projektet var att skapa ett fungerande spel där spelarens val påverkar vad som händer och där olika funktioner samverkar. Programmet behöver exempelvis hålla reda på spelarens hälsa och mängden guld, kontrollera om spelaren har besegrat vissa fiender och avgöra om spelaren får gå vidare till slutbossen.

Det behövdes också ett sätt att hantera felaktiga menyval och förhindra att spelaren får samma skatt flera gånger. Butiken behövde kunna hantera köp, kontrollera om spelaren har tillräckligt med guldmynt och se till att spelarens hälsa inte överstiger den maximala nivån.

Projektets huvudsakliga problem kan därför beskrivas som att konstruera en konsolbaserad applikation där flera funktioner samverkar och där spelets regler följs under hela spelomgången.

1.3 Syfte

Syftet med projektet är att utveckla ett textbaserat RPG- spel i C# som använder kunskaper från kursen *Programmering i C#.NET.* Genom att utveckla spelet vill jag få praktisk erfarenhet av att strukturera programkod, hantera användarinmatning och skapa funktioner som samverkar i en större applikation.

Projektet syftar även till att ge en bättre förståelse för hur villkorssatser, loopar och variabler kan användas för att styra ett programs beteende.

1.4 Mål och avgränsningar

Målet är att skapa ett spel som kan startas, spelas och avslutas via konsolens menyer. Spelaren ska kunna skapa en karaktär genom att ange ett namn, utforska olika platser, strida mot fiender, samla skatter och köpa föremål.

Spelet innehåller en mörk skog, ett övergivet slott och en slutboss i form av en drake. Spelaren kan köpa ett bättre svärd som ökar skadan i strider och hälsodrycker som återställer hälsa. För att nå slutbossen så måste spelaren först besegra fienden i skogen och den onda riddaren i slottet.

Projektet är avgränsat till ett textbaserat konsolprogram. Fokus ligger på spelets logik och funktioner snarare än på avancerad grafik eller ett grafiskt användargränssnitt.

1.5 Författarens bidrag

Hela arbetet är skapat av mig (Lina Wiklund).

2 Bakgrundsmaterial

2.1 C#

C# är ett programmeringsspråk som används för att utveckla olika typer av applikationer. Språket kan bland annat användas för konsolprogram, webbapplikationer och program med grafiska användargränssnitt.

I projektet *Det Försvunna Kungariket* används C# för att skapa spelets logik och hantera spelarens val. Programmet avgör exempelvis vilken plats spelaren kan besöka, hur strider genomförs och hur mycket hälsa och guldmynt som spelaren har kvar.

C# gör det möjligt att dela upp programmet i olika instruktioner som utförs beroende på vilka villkor som är uppfyllda. Detta är viktigt i ett spel där olika händelser inträffar beroende på spelarens handlingar.

2.2 .NET

.NET är en utvecklingsplattform som används för att skapa och köra applikationer. Plattformen innehåller bland annat bibliotek och verktyg som programmeraren kan använda för att utveckla program.

I detta projekt används .NET för att köra konsolprogrammet som har skrivits i C#. Genom funktioner i .NET kan programmet exempelvis visa text i konsolen, läsa in spelarens val och göra korta pauser mellan olika händelser.

Ett exempel från spelet se figur 1.

Figur 1

```
// Huvudmeny
Console.WriteLine("\n================================================");
Console.WriteLine("\n    Välkommen till Det Försvunna Kungariket!      ");
Console.WriteLine("\n================================================");

Console.WriteLine("\n");
Console.WriteLine("1. Starta spelet!");
Console.WriteLine("2. Avsluta");

Console.WriteLine();
Console.WriteLine("Välj ett alternativ (1-2): ");
```

Instruktionen använder konsolens funktioner för att visa ett meddelande för spelaren. På så sätt får spelaren information om vad som händer i spelet.

2.3 Variabler och datatyper

Variabler används för att lagra information som programmet behöver under körningen. I C# kan olika datatyper användas beroende på vilken information som ska lagras, exempelvis heltal och text [1]. I spelet används

flera olika variabler för att hålla reda på spelarens egenskaper och spelets tillstånd.

Ett exempel från programkoden se figur 2.

Figur 2

```
// Läser in spelarens namn och sparar det i variabeln 'playerName'.
string playerName = Console.ReadLine() ?? "";
// Sätt spelarens hälsa till 100.
int health = 100;
// Spelarens guld.
int gold = 0;
// Om spelaren har ett bättre svärd
bool hasBetterSword = false;
// Håller reda på om skogen är klar.
bool forestCompleted = false;
// Håller reda på om slottet är klar.
bool castleCompleted = false;
// Håller reda på om skatten är hittad i skogen.
bool forestTreasureFound = false;
// Håller reda på om skatten är hittad i slottet.
bool castleTreasureFound = false;
```

Variabeln health lagrar spelarens hälsa, medan gold lagrar spelarens guldmynt. Datatypen int används för heltal. Variabler av typen boolean har också använts. Ett exempel är hasBetterSword. Den här används för att kontrollera om spelaren har köpt ett bättre svärd.

Variablerna förändras när spelaren utför olika handlingar. Om spelaren exempelvis hittar en skattkista ökar mängden guld. Om spelaren blir attackerad så minskar hälsan.

2.4 Villkorssatser

Villkorssatser används för att låta programmet fatta beslut. Med if, else if och else kan programmet utföra olika instruktioner beroende på om ett villkor är uppfyllt.

I spelet används villkorssatser för att hantera menyval, kontrollera om spelaren har tillräckligt med guldmynt och avgöra om en fiende har besegrats.

Ett exempel från butiken se figur 3.

Figur 3

```
if (shopChoise == "1")
{
    if (hasBetterSword)
    {
        Console.WriteLine("\nDu har redan det bästa svärdet!");
    }
    else if (gold >= 20)
    {
        gold -= 20;
        hasBetterSword = true;

        Console.WriteLine("\nDu köpte ett bättre svärd!");
        Console.WriteLine("Ditt nya svärd gör 40 skada.");

        Console.WriteLine($"\nDu har {gold} guldmynt kvar.");
    }
    else
    {
        Console.WriteLine("\nDu har inte tillräcklig med gultmynt!");
        Console.WriteLine("Du behöver 20 guldmynt för att köpa svärdet.");
    }
    Console.WriteLine("\nTryck på valfri tangent för att fortsätta.");
    Console.ReadKey(true);
}
```

Här kontrollerar programmet om spelaren redan har det bättre svärdet, annars kollar programmet om spelaren har tillräckligt med guldmynt för att köpa det bättre svärdet. Om villkoret är uppfyllt så dras 20 guldmynt från spelarens saldo och variabeln hasBetterSword ändras till true. Om spelaren däremot saknar tillräckligt med guldmynt så visas det i stället ett meddelande.

Villkorssatser gör det möjligt att implementera spelets regler och förhindra att spelaren genomför köp som inte är tillåtna.

2.5 Loopar

Loopar används när en del av programmet behöver upprepas. I spelet används while-loopar för att hålla menyer och strider i gång tills ett visst villkor inte längre är uppfyllt [2].

Ett exempel från spelet se figur 4.

Figur 4

```
// Fortsätt visa spelmenyn så länge spelet är igång.
while (playing)
{
    Console.Clear();
    // Visa spelets meny.
    Console.WriteLine("\n==================================================");
    Console.WriteLine("\n                    SPELMENY                     ");
    Console.WriteLine("\n==================================================");

    Console.WriteLine("\n1. Utforska kungariket");
    Console.WriteLine("2. Visa karaktärens information");
    Console.WriteLine("3. Besök butiken");
    Console.WriteLine("4. Avsluta spelet");

    Console.Write("\nVad vill du göra? (1-4): ");

    // Läser in spelarens val och sparar det i variabeln 'playerChoice'.
    string playerChoice = Console.ReadLine() ?? "";
```

Så länge playing är true så fortsätter loopen att köras. När spelaren väljer att avsluta spelet ändras variabeln till false, vilket gör att loopen avslutas.

Det finns även loopar för utforskning av kungariket och för strider mot fiender. Dessa loopar gör att spelaren kan välja flera handlingar innan hen lämnar en plats eller striden tar slut.

2.6 Användarinmatning och utskrifter

Eftersom spelet är textbaserat sker kommunikationen med spelaren genom konsolen. Programmet använder Console.WriteLine() för att visa information och Console.ReadLine() för att läsa in text från användaren [3].

Ett exempel på inläsning se figur 5.

Figur 5

```
string fightChoice = Console.ReadLine() ?? "";
```

Här läser programmet in spelarens val och lagrar det i variabeln fightChoice. Om inläsningen ger null så används i stället en tom sträng.

Programmet jämför sedan spelarens inmatning med olika alternativ. Om spelaren exempelvis skriver 1 kan programmet köra striden.

Den här lösningen gör att spelaren kan styra spelet genom att skriva in siffror i menyerna.

2.7 Logiska uttryck

Logiska uttryck används för att kombinera eller förändra villkor. I C# kan bland annat operatorerna &&, || och ! användas för detta []4.

I spelet används exempelvis följande villkor för att kontrollera om spelaren får möta slutbossen. Se figur 6.

Figur 6

```
// Koll om spelaren har gjort skogen och slottet först.
if (!forestCompleted || !castleCompleted)
{
    Console.WriteLine("\nDu har hittat vägen till kungarikets mörkaste plats...");
    Console.WriteLine("Men en mäktig kraft blockerar vägen!");

    Console.WriteLine("\nDu måste besegra monstret i skogen");
    Console.WriteLine("och den onda riddaren i slottet först.");

    Console.WriteLine("\nTryck på valfri tangent för att fortsätta.");
    Console.ReadKey(true);
}
```

Variablerna forestCompleted och castleCompleted anger om spelaren har besegrat fienderna i skogen respektive slottet.

Operatorn ! betyder att villkoret vänds. Operatorn ! betyder att villkoret vänds. Operatorn || betyder att minst ett av villkoren behöver vara sant för att hela uttrycket ska vara sant [4].

Det innebär att meddelandet visas om spelaren ännu inte har klarat skogen eller slottet. För att få möta slutbossen måste båda områdena vara avklarade.

2.8 Sammanfattning av teorin

Spelet använder flera grundläggande delar av C# och .NET för att skapa en interaktiv applikation. Variabler lagrar information om spelaren, villkorssatser styr vilka handlingar som är tillåtna och loopar gör det möjligt att upprepa menyer och strider.

Genom att kombinera dessa tekniker kan programmet hantera spelarens val, genomföra strider, uppdatera hälsa och guldmynt samt kontrollera när spelaren får gå vidare i spelet.

3 Metod

3.1 Arbetsprocess

Arbetet med *Det Försvunna Kungariket* genomfördes genom att jag utvecklade spelet stegvis i C# i Visual Studio. Jag valde att dela upp utvecklingen i mindre delar för att kunna fokusera på en funktion i taget. På så sätt kunde jag bygga vidare på tidigare delar när jag skulle lägga till nya funktioner.

Jag började med att skapa grunden för konsolprogrammet. Därefter utvecklade jag spelets menyer och funktioner för att ta emot spelarens val. När grundstrukturen var på plats så fortsatte jag med att lägga till karaktärens egenskaper, spelvärlden, striderna, skattkistorna, butiken och slutligen spelets slutstrid.

Under utvecklingen behövde jag tänka på hur de olika funktionerna skulle hänga ihop. Exempelvis måste spelarens guldmynt uppdateras när spelaren hittar en skatt eller besegrar en fiende. Samma mängd guldmynt måste sedan användas när spelaren köper föremål i butiken. På motsvarande sätt behöver spelarens hälsa påverkas av fiendernas attacker och återställas när spelaren använder en hälsodryck.

En viktig del av arbetet var därför att skapa en struktur där spelets olika delar kunde påverka samma speldata. Jag använde variabler för att spara information om spelarens tillstånd och villkorssatser för att bestämma vad som skulle hända beroende på spelarens val och aktuella situation.

3.2 Utveckling av spelets grundstruktur

Det första steget var att skapa en fungerande grund för programmet. Jag började med en startmeny där spelaren kan välja att starta spelet eller avsluta programmet. Syftet med startmenyn var att ge spelaren en tydlig början och göra det möjligt att välja om spelet skulle startas.

Därefter skapade jag en funktion för att låta spelaren ange sitt karaktärsnamn. Jag använde konsolens in- och utmatning för att visa frågor och läsa in spelarens svar. När spelaren angett sitt namn kan spelet gå vidare till huvudmenyn.

Jag utvecklade sedan huvudmenyn, där spelaren kan välja mellan att utforska kungariket, visa karaktärsinformation, besöka butiken eller avsluta spelet. Menyn fungerar som spelets centrala utgångspunkt. Därifrån kan spelaren välja vilken aktivitet som ska genomföras.

För att spelet skulle kunna fortsätta tills spelaren väljer att avsluta det eller att spelet tar slut så använde jag en while-loop med en boolean variabel. Variabeln playing används för att hålla reda på om spelet fortfarande ska vara aktivt. Så länge variabeln har värdet true fortsätter programmet att visa huvudmenyn och hantera spelarens val.

Jag använde även villkorssatser för att koppla varje menyval till rätt funktion. När spelaren exempelvis väljer att visa karaktärsinformation ska spelet visa uppgifter om karaktären, medan ett val att besöka butiken leder till butikens meny. Denna struktur gör att spelaren kan återvända till huvudmenyn efter att ha genomfört olika aktiviteter.

3.3 Variabler och hantering av speldata

En central del av utvecklingen var att bestämma vilken information spelet behövde spara under spelomgången. Jag använde variabler för att hålla reda på spelarens hälsa, guldmynt, utrustning och framsteg.

Variabeln health används för att representera spelarens hälsa. Hälsan börjar på 100 poäng och kan minska när spelaren blir attackerad av fiender. Om spelaren köper och använder en hälsodryck kan hälsan i stället öka, men den kan inte överstiga det maximala värdet på 100.

Variabeln gold används för att hålla reda på spelarens guldmynt. Spelaren kan samla guldmynt genom att besegra fiender och hitta skattkistor. Guldmynten används sedan som betalningsmetod i butiken. När spelaren köper ett föremål minskas värdet i variabeln med föremålets pris.

För att hålla reda på om spelaren har köpt ett bättre svärd så använde jag en boolean variabel has BetterSword. Den gör det möjligt för spelet att kontrollera om spelaren redan har köpt svärdet. Om spelaren har det ska spelet inte låta spelaren köpa samma uppgradering igen.

Jag skapade också variablerna forestCompleted och castleCompleted för att hålla reda på om spelaren har klarat skogen respektive slottet. Dessa variabler används bland annat för att avgöra om spelaren får möta spelets slutboss.

Slutligen använder spelet variablerna forestTreasureFound och castleTreasureFound för att registrera om spelaren redan har hittat skattkistan i respektive område. Detta förhindrar att samma skatt kan ge guldmynt upprepade gånger.

Genom att använda dessa variabler kunde jag låta spelet komma ihåg viktiga händelser under spelomgången. Informationen påverkar senare val och gör att spelet reagerar olika beroende på vad spelaren redan har gjort.

3.4 Utveckling av spelvärlden och utforskningsmenyn

När spelets grundstruktur var klar utvecklade jag utforskningsmenyn. Där kan spelaren välja mellan den mörka skogen, det övergivna slottet, slutbossen och att återvända till huvudmenyn.

Jag delade upp spelvärlden i olika områden för att skapa en tydligare spelstruktur. Skogen och slottet innehåller varsin fiende och skattkista, medan slutbossen fungerar som spelets avslutande utmaning.

Varje område har en egen meny med alternativ som spelaren kan välja mellan. I skogen kan spelaren exempelvis utforska området, leta efter en skattkista eller gå tillbaka. I slottet finns motsvarande alternativ för att utforska, hitta skattkista och lämna området.

För att hantera de olika alternativen läser programmet in spelarens val och jämför det med de alternativ som finns i menyn. Beroende på valet utförs den tillhörande delen av programmet.

Jag behövde också se till att spelaren kunde lämna ett område och återvända till menyerna utan att hela programmet avslutades. Detta är viktigt eftersom spelaren ska kunna röra sin mellan spelets olika delar och exempelvis besöka butiken efter att ha samlat guldmynt.

3.5 Utveckling av stridssystemet

En viktig funktion i spelet är stridssystemet. Jag utvecklade striderna så att spelaren möter en fiende, väljer om den ska attackera eller fly och får resultatet av handlingen presenterat i konsolen.

I den mörka skogen möter spelaren ett monster med 50 hälsopoäng. I det övergivna slottet möter spelaren en ond riddare med 60 hälsopoäng. Fienderna har olika mängd hälsa och gör olika mycket skada, vilket ger områdena skilda svårighetsgrader.

Spelarens grundskada är 20 poäng per attack. Om spelaren köper det bättre svärdet så ökar skadan till 40 poäng. Därför behöver stridssystemet kontrollera värdet på hasBetterSword innan det bestämmer hur mycket skada spelarens attack ska göra.

Under en strid minskar fiendens hälsa när spelaren attackerar. Om fienden fortfarande har hälsa kvar kan den attackera spelaren och minska spelarens hälsa. I skogen gör monstret 10 poäng i skada per attack, medan riddaren gör 15 poäng. Striden fortsätter tills fienden besegrats, spelaren har besegrats eller spelaren väljer att fly.

När en fiende har besegrats uppdateras spelets information. Spelaren får guldmynt och området registreras som avklarat. Om spelaren däremot förlorar all sin hälsa avslutas spelomgången.

Jag lade också till möjligheten att fly från en strid. Det ger spelaren ett alternativ till att fortsätta slåss och gör att spelaren själv kan påverka hur en strid utvecklas. Om spelaren flyr utan att besegra fienden får spelaren ingen belöning för segern.

Stridssystemet är beroende av flera variabler och villkor som samverkar. Därför är det viktigt att hälsan uppdateras korrekt efter varje attack och att spelet kontrollerar om någon av parterna har besegrats innan striden fortsätter.

3.6 Skattkistor och belöningar

För att göra utforskningen mer betydelsefull så lade jag till skattkistor i både skogen och slottet. Skattkistorna ger spelaren möjlighet att samla ytterligare guldmynt utöver de belöningar som erhålls efter strider.

Skatten i skogen ger 20 guldmynt och skatten i slottet ger 30 guldmynt. När spelaren väljer att leta efter en skatt så kontrollerar programmet om skatten redan har hittats.

Om skattkistan inte har hittats får spelaren belöningen och den tillhörande boolean variabeln ändras till true. Om spelaren försöker hitta samma skatt igen kan programmet kontrollera variabeln och undvika att dela ut belöningen på nytt.

Denna funktion behövdes för att förhindra att spelaren kunde samla obegränsat med guldmynt genom att upprepa samma handling. Den gör också så att spelet kan komma ihåg vad spelaren redan har gjort i varje område.

3.7 Utveckling av butiken

Efter att ha skapat funktionerna för strider och belöningar så utvecklade jag en butik där spelaren kan använda sina guldmynt för att köpa uppgraderingar och återställa hälsa.

Butiken innehåller två köpbara föremål: ett bättre svärd och en hälsodryck. Det bättre svärdet kostar 20 guldmynt och öka spelarens attackskada från 20 till 40 poäng. Hälsodrycken kostar 15 guldmynt och återställer 30 hälsopoäng.

När spelaren öppnar butiken visas den aktuella mängden guldmynt tillsammans med de tillgängliga alternativen. Efter att spelaren har valt ett föremål kontrollerar programmet om köpet är möjligt.

Vid köp av svärdet så kontrollerar programmet först om spelaren redan äger det. Om svärdet redan är köpt ska spelaren få ett meddelande om detta. Om spelaren inte äger svärdet kontrolleras om spelaren har tillräckligt med guldmynt. Vid ett godkänt köp så dras 20 guldmynt från spelarens saldo och variabeln hasBetterSword ändras till true.

Vid köp av en hälsodryck så kontrollerar programmet om spelarens hälsa redan är maximal. Om hälsan är 100 behöver spelaren inte köpa en dryck. Om spelaren har lägre hälsa så kontrolleras i stället om det finns tillräckligt med guldmynt. När köpet genomförs så dras 15 guldmynt från spelarens saldo och spelarens hälsa ökar med 30 poäng.

Jag lade till en kontroll som säkerställer att hälsan inte överstiger 100 poäng. Om en dryck skulle öka hälsan över maxvärdet sätts hälsan tillbaka till 100. På så sätt kan spelaren inte få mer hälsa än spelets regler tillåter.

Butiken behövde även hantera situationer där spelaren saknar tillräckligt med guldmynt. I sådana fall ska köpet inte genomföras och spelaren får i stället ett meddelande om att guldmynten inte räcker.

Under utvecklingen behövde jag se till att butikens olika villkor fungerade tillsammans. Det var särskilt viktigt att möjligheten att köpa en hälsodryck inte påverkades av om spelaren redan hade köpt svärdet. Varje köp måste kontrolleras utifrån sina egna regler.

3.8 Utveckling av slutbossen

Slutbossen fungerar som spelets sista utmaning och är utformad för att spelaren först ska behöva utforska de andra områdena.

För att få möta slutbossen måste spelaren ha klarat både skogen och slottet. Programmet kontrollerar därför variablerna forestCompleted och castleCompleted innan striden kan starta. Om något av områdena inte är avklarat så får spelaren inte gå vidare till slutstriden.

När villkoren är uppfyllda kan spelaren möta draken, som har 100 hälsopoäng. Spelarens skada beror på om det bättre svärdet har köpts, medan draken gör 20 poäng i skada per attack.

Precis som i de andra striderna kan spelaren välja att attackera eller fly. Striden fortsätter tills draken besegrats, spelaren förlorar sin hälsa eller om spelaren väljer att fly.

När draken besegras avslutas spelet med en seger. Om spelaren besegras avslutas spelomgången också, men med ett annat resultat. Om spelaren flyr kan den återvända till spelet och försöka möta draken igen senare.

Genom att lägga till kravet att både skogen och slottet måste vara avklarade skapade jag en progression där spelaren behöver utforska spelvärlden innan spelets slutmål kan nås.

3.9 Testning och felsökning

Testning var en viktig del av utvecklingsarbetet eftersom flera av spelets funktioner påverkar varandra. Ett köp i butiken förändrar exempelvis spelarens guldmynt och kan även påverka resultatet i framtida strider. På samma sätt påverkar resultatet av en strid både spelarens hälsa, guldmynt och framsteg.

Jag testade spelet genom att köra programmet och prova de olika menyerna och funktionerna. Målet var att kontroller att spelaren kunde starta spelet, skapa en karaktär, välja område, genomföra strider, samla belöningar, handla i butiken och försöka besegra slutbossen.

Jag behövde även kontrollera situationer där ett val inte borde genomföras. Exempel på detta är när spelaren försöker köpa ett föremål utan tillräckligt

med guldmynt, köpa ett svärd som redan ägs eller använda en hälsodryck när hälsan är redan maximal.

Ett exempel på felsökning gällde butikens villkor. Jag upptäckte att butikens funktioner behövde struktureras så att spelaren kunde fortsätta använda butiken även efter att ha köp svärdet. Jag ändrade därför villkorsstrukturen så att kontrollen av svärdet hanterades i samband med själva köpet, i stället för att hindra andra butiksfunktioner.

Testningen gav mig möjlighet att hitta och rätta problem under utvecklingen. Det hjälpte mig också att kontrollera att de olika funktionerna fungerade tillsammans och att spelet gick att spela från början till slut.

4 Konstruktion

4.1 Programmeringsmiljö och gränssnitt

Spelet är utvecklat i C# och använder konsolen för att visa information och ta emot spelarens inmatning. Gränssnittet består av text, rubriker, menyval och meddelanden som beskriver vad som händer i spelvärlden.

Jag valde en textbaserad lösning där spelaren skriver in siffror för att välja mellan de olika alternativen. Programmet läser in valet och använder det för att avgöra vilken del av spelet som ska köras.

För att göra menyerna tydligare använder programmet bland annat Console.WriteLine() för att visa text och Console.Write() för att visa frågor där spelaren ska skriva sitt svar. Metoden Console.ReadLine() används för att läsa in text från spelaren [3].

Programmet använder även Console.Clear() för att rensa konsolfönstret innan nya menyer visas. Det gör att spelet inte hela tiden fyller skärmen med innehåll från tidigare menyer. I vissa delar används Thread.Sleep() för att skapa en kort paus innan nästa meddelande visas. Funktionen används exempelvis när spelaren går in i ett område eller när en fiende dyker upp.

En fördel med konsolgränssnittet är att det gör det möjligt att fokusera på spelets logik utan att behöva utveckla grafik.

4.2 Variabler och datatyper

För att spelet ska kunna komma ihåg spelarens egenskaper och handlingar används variabler. Variablerna deklareras efter att spelaren har valt att starta spelet och angett sitt karaktärsnamn. De används sedan under resten av spelomgången.

En av variablerna är playerName, som är av datatypen string. Den lagrar namnet som spelaren skriver in och används senare när spelet visar information om karaktären och beskriver händelser i spelet.

Variabeln health är av datatypen int och representerar spelarens hälsa. När spelet startar sätts värdet till 100. Under strider minskar värdet när fiender attackerar spelaren. Om spelaren köper en hälsodryck ökar värdet igen, men programmet kontrollerar att hälsan inte överstiger 100.

Variabeln gold, som också är av datatypen int, används för att lagra spelarens guldmynt. Spelaren börjar med noll guldmynt och kan samla mer genom att hitta skattkistor och besegra fiender. Guldet används därefter som betalningsmedel i butiken.

Programmet använder även flera variabler av datatypen bool. En boolesk variabel kan ha värdet true eller false och passar därför för att registrera om något har hänt eller om ett visst villkor är uppfyllt [1].

Följande boolean värden används i spelet:

- playing- anger om spelets huvudloop ska fortsätta.
- hasBetterSword- anger om spelaren har köpt det bättre svärdet.
- forestCompleted- anger om spelaren har besegrat monstret i skogen.
- castleCompleted- anger om spelaren besegrat den onda riddaren i slottet.
- forestTreasureFound- anger om skattkistan i skogen har hittats.
- castleTreasureFound- anger om skattkistan i slottet har hittats.

Dessa variabler används för att styra spelets logik.

Genom att kombinera olika datatyper så kan programmet både lagra numeriska värden och hålla reda på händelser och villkor som påverkar spelets fortsättning.

4.3 Menysystem och villkorssatser

Spelets menyer styrs huvudsakligen av villkorssatser. När spelaren väljer ett alternativ så läser programmet in valet och jämför det med de olika alternativen.

I huvudmenyn används exempelvis en struktur med if, else if och else.

Principen kan illustreras med följande kodutdrag från huvudmenyn. Se figur 7 och 8.

Figur 7

```
// Läser in spelarens val och sparar det i variabeln 'playerChoice'.
string playerChoice = Console.ReadLine() ?? "";

// Hantera spelarens val.
if (playerChoice == "1")
{
    Console.Clear();
    Console.WriteLine("\nDu börjar utforska det försvunna kungariket...");

    Thread.Sleep(2000);
```

Figur 8

```
else if (playerChoice == "2")
{
    Console.WriteLine("\n===== DIN KARAKTÄR =====");
    Console.WriteLine($"Namn: {playerName}");
    Console.WriteLine($"Hälsa: {health}");
    Console.WriteLine($"Guldmynt: {gold}");
}
else if( playerChoice == "3")
{
    Console.Clear();

    Console.WriteLine("========================================");
    Console.WriteLine("                BUTIKEN                 ");
    Console.WriteLine("========================================");

    Console.WriteLine($"\nDu har {gold} guldmynt");


    Console.WriteLine("\n1. Köp ett bättre svärd - 20 guldmynt");
    Console.WriteLine("      Skadan ökar från 20 till 40.");
    Console.WriteLine("2. Köp en hälsodryck - 15 guldmynt");
    Console.WriteLine("     Återställer 30 hälsopoäng.");
    Console.WriteLine("3. Gå tillbaka");

    Console.Write("\nVad vill du göra? ");
    string shopChoise = Console.ReadLine() ?? "";
```

Koden visar hur spelarens inmatning jämförs med olika alternativ. Samma princip används i andra delar av spelet.

Villkorssatserna används även för att kontrollera spelets regler. Exempelvis i butiken så kontrollerar programmet om spelaren har tillräckligt med guldmynt för ett köp.

En viktig del av lösningen är att villkoren placeras i rätt ordning och på rätt nivå i programstrukturen. Om ett villkor placeras övergripande så kan det påverka flera alternativ som egentligen borde fungera oberoende av varandra.

4.4 Loopar och navigering mellan menyer

Spelet använder while-loopar för att upprepa olika delar av programmet. Looparna gör att spelaren kan använda menyerna flera gånger utan att programmet behöver avslutas efter varje val [2].

Den viktigaste loopen styrspelets huvudmeny:

Bool playing = true;

While (playing)

{

// Visa huvudmenyn.

// Läs in spelarens val.

```
        // Hantera det valda alternativet.

}
```

Så länge playing har värdet true fortsätter programmet att visa huvudmenyn och hantera spelarens val. När spelaren väljer att avsluta så ändras variabeln till false, vilket gör att loopen avslutas.

Det finns även loopar för utforskningsmenyn och för de olika områdena. Variabeln exploringKingdom används för att hålla utforskningsmenyn aktiv, medan exploringForest och exploringCastle används för att styra respektive områdes meny.

Denna konstruktion innebär att spelaren kan gå in i ett område, välja en aktivitet och därefter återvända till områdets meny eller lämna området helt. När spelaren väljer att gå tillbaka ändras den aktuella booleska variabeln till false, vilket avslutar just den loopen.

Striderna använder också loopar. I skogen och slottet fortsätter striden så länge striden är aktiv och både spelaren och fienden har hälsa kvar. I bossstriden används motsvarande princip.

Genom att använda olika booleska variabler för olika loopar kan programmet styra flera nivåer av navigering. Det gör det möjligt att avsluta en strid utan att nödvändigtvis avsluta hela spelet, eller att lämna ett område och återvända till utforskningsmenyn.

5 Resultat

Resultatet blev ett textbaserat rollspel i C# där spelaren kan utforska olika områden, slåss mot fiender, samla guldmynt och köpa utrustning. Spelet innehåller menyer, strider, skatter och en slutboss som blir tillgänglig när spelaren har klarat skogen och slottet.

Under utvecklingen har jag använt variabler, villkor och loopar för att styra spelets funktioner och hålla reda på spelarens hälsa, guldmynt och framsteg. Spelet innehåller flera funktioner som samverkar för att skapa en enkel spelupplevelse.

Resultatet visar hur grundläggande programmering i C# kan användas för att skapa ett interaktivt spel med flera funktioner och en tydlig målsättning.

6 Slutsatser

Arbetet med *Det Försvunna Kungariket* har gett mig en bättre förståelse för hur man kan använda C# för att skapa ett spel med olika funktioner och valmöjligheter. Jag har fått öva på att använda variabler, villkor och loopar för att styra spelets gång och hålla reda på spelarens hälsa, guldmynt och framsteg.

En slutsats jag drar är att det är viktigt att planera och strukturera koden redan från början. När spelet växte blev det fler delar att hålla reda på. Om jag skulle utveckla spelet vidare skulle jag dela upp koden i flera egna metoder, exempelvis för menyer, strider och butiken. Det skulle göra programmet lättare att läsa, förstå och felsöka.

Sammanfattningsvis så har projektet gett mig praktisk erfarenhet av programmering och visat hur grundläggande programmeringskunskaper kan användas för att skapa ett fungerande spel. Jag ser också flera möjligheter till att utveckla spelet vidare, exempelvis genom att förbättra strukturen och lägga till fler funktioner.

Källförteckning

[1] W3schools, ”C# Variables” https://www.w3schools.com/cs/cs_variables.php Hämtad: 2026-10-28

[2] W3schools, ”C# While Loop” https://www.w3schools.com/cs/cs_while_loop.php Hämtad: 2026-10-28

[3] W3schools, ”Get User Input” https://www.w3schools.com/cs/cs_user_input.php Hämtad: 2026-10-28

[4] W3schools, ”C# Logical Operators” https://www.w3schools.com/cs/cs_operators_logical.php Hämtad: 2026-10-28